

# SC08 | Machine Predictive Maintenance & Fault Priority System

**Difficulty:** Advanced capstone   **Core techniques:** ANN + Fuzzy Logic   **Team:** 3-4 students

## 1. Project overview

Create a working decision-support product that converts **vibration, temperature, RPM, load and severity** into **fault likelihood and maintenance priority**. This is not a generic machine-learning exercise: the product must model uncertainty, trade-offs, and imperfect real-world conditions through **ANN + Fuzzy Logic**. The final application must be runnable by a non-technical user, explain its result visually, and demonstrate a measurable improvement over a rigid rule-only approach.

## 2. Constraints and non-negotiable requirements

|                                                           |                                                                                     |                                                                                  |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Team</b><br>3-4 students                               | <b>Experimental data</b><br>10,000+ cited records or documented scenarios           | <b>Product</b><br>Runnable UI with input, processing, output and visual evidence |
| <b>Repository</b><br>Continuous, attributable Git commits | <b>Deployment/UI</b><br>Publicly accessible hosted product link is mandatory for M2 | <b>Submission</b><br>Report, 5-minute video, presentation and individual viva    |

## 3. Why this is a Soft Computing capstone

- The problem contains uncertainty, non-linear behaviour, competing goals, or incomplete information.
- You must expose the important algorithmic elements rather than only calling a library function.
- A baseline is compulsory: crisp sensor thresholds. Your evidence must show what the Soft Computing solution improves.

## 4. Product and system architecture

**Pipeline:** data/scenario collection → validation and preprocessing → baseline → ANN + Fuzzy Logic model → decision/optimised output → evaluation dashboard.

Recommended stack: Python, NumPy/Pandas, Matplotlib or Plotly, Streamlit, and GitHub. Use scikit-learn only for a comparison baseline where relevant; it is not a substitute for explaining the course algorithm.

## 5. Data and experimental strategy

**Required source:** cited 10,000+ sensor readings or documented augmentation. Document dataset name, URL, licence, original size, features/targets, all preprocessing, and any scenario-generation assumptions. The minimum is 10,000 records or realistic test scenarios. Do not invent a citation: state clearly what is real and what is simulated.

## 6. Milestone 1 - Working Product V1 (mid-semester)

**Objective:** Create a locally working, evidence-backed first version of the product. M1 is a build milestone, not a documentation milestone.

Build a fault-risk or priority MVP with sensor input, baseline and error plots.

### Functional requirements:

- Accept the stated project inputs, validate them, run the core algorithm, and return the required output.
- Display both the Soft Computing result and the baseline result in a way a user can understand.

- Save reproducible results and at least two clear visualisations.
- Fuzzy component: define linguistic variables, membership shapes/ranges, fuzzification, an explicit rule base, aggregation, defuzzification, and membership/rule visualisations.
- Neural component: document inputs/targets, normalisation, architecture, activation, forward pass, learning/error, train/test split, convergence, and prediction-quality plots.

#### **M1 submission evidence:**

- Repository created early, with README, requirements, source/app folders, data citation, and continuous commits.
- At least 10,000 records/scenarios documented and a reproducible preprocessing or generation step.
- Interim report with problem, input/output specification, architecture, algorithm explanation, MVP screenshots, results, limitations and next steps.
- A live M1 demo: input → processing → output. Documentation-only work is not accepted.

## **7. Milestone 2 - Enhanced Product V2 (end-semester)**

**Objective:** Convert the M1 prototype into a complete, reliable and publicly hosted product with advanced experimentation.

Combine ANN risk with fuzzy severity; tune, compare and test noisy/extreme sensor conditions.

#### **Functional and technical requirements:**

- Complete the advanced/hybrid component and make the final application runnable from README instructions.
- Run parameter experiments rather than a single best run. Record settings, results, interpretation and selected configuration.
- Compare baseline, M1 approach, and advanced M2 approach where applicable, using F1, ROC-AUC and priority consistency.
- Deliberately test edge cases: noise, extreme values, contradictory inputs, unseen cases, or difficult constraints.
- Deploy the final UI on an approved public platform. Localhost-only demonstrations are not accepted.
- Publish final plots/tables, report, ~5-minute demo video, presentation material, and team contribution details.

## **8. Required repository evidence**

- README with team, problem, architecture, dataset citation, setup/run commands, results, deployment link and limitations.
- src/ for reusable algorithms; app/ for runnable product; results/ for labelled outputs; report/ for reports; data/README.md for source information.
- Meaningful commits throughout the semester, such as “Implement fuzzy rules”, “Add GA mutation”, or “Evaluate baseline”. A final ZIP/code dump does not earn full Git marks.

## **9. Evaluation evidence (50 marks)**

|                                                                                                      |                                                                                                                |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>GitHub - 10</b><br>Structure, code quality, continuous commits and contribution.                  | <b>Report - 10</b><br>Problem, data citation, method, implementation, experiments and results.                 |
| <b>Video - 10</b><br>Problem, concept explanation, product demo, comparison and clarity.             | <b>Deployment &amp; UI - 10</b><br>Public hosted link, working flow, usability, visualisation and reliability. |
| <b>Viva &amp; Presentation - 10</b><br>Technique understanding, own code, results and communication. | <b>Total - 50</b><br>Viva marks are individual; other artefacts may be team-level.                             |

## Final reminder

This is an engineering product capstone. A strong submission makes it easy for a faculty member to run the project, inspect the evidence, compare it with the baseline, and ask each student about the part they built.